

Συστήματα Ευφών Πρακτόρων - 1η Ομαδική Εργασία



Μάθημα: Συστήματα Ευφών Πρακτόρων
Διδάσκων: Γεώργιος Βούρος

Αρ.	Ονοματεπώνυμο	A.M.	E-mail
1	Δαμιανός Ζούμπος	E22056	d.zoumpos04@gmail.com
2	Γεώργιος Αζαμ	E22003	george.azam11@gmail.com
3	Άννα-Μαρία Νάιντενοβα	E22117	annamaria.ndn@gmail.com

Ημερομηνία Παράδοσης:
Ιανουάριος 2026

Περιεχόμενα

1	Εισαγωγή και Περιγραφή Εργασίας	2
2	Μοντελοποίηση Περιβάλλοντος	2
2.1	Πλέγμα και αρχική θέση πράκτορα	2
2.2	Αντικείμενα, στατικές θέσεις και τοίχοι	3
2.3	Ενέργειες πράκτορα (υλοποιημένες και αποκλίσεις)	4
2.4	Περιορισμοί και προϋποθέσεις εργασιών	4
3	Σχεδιασμός Πράκτορα BDI	5
3.1	Beliefs (Πεποιθήσεις)	5
3.2	Goals (Στόχοι)	6
3.3	Επιλογή στόχου βάσει Αναμενόμενης Χρησιμότητας	7
3.4	Σχεδιασμός Πλάνων Ενεργειών	7
4	Αποτελέσματα και Συζήτηση	9
4.1	Κατάσταση εκτέλεσης και διαθέσιμα ευρήματα	9
4.2	Ζητήματα υλοποίησης που επηρέασαν την εκτέλεση	9

1 Εισαγωγή και Περιγραφή Εργασίας

Στην παρούσα αναφορά παρουσιάζουμε τον σχεδιασμό, την υλοποίηση και την πειραματική αξιολόγηση ενός πράκτορα πρακτικού συλλογισμού (Practical Reasoning Agent) στο πλαίσιο του μαθήματος *Συστήματα Ευφυνών Πρακτόρων*.

Ο πράκτορας βασίζεται στο μοντέλο BDI (Beliefs-Desires-Intentions) και λειτουργεί σε περιβάλλον πλέγματος 5×5 . Οι βασικοί στόχοι (Desires) είναι:

- να βάψει το τραπέζι (Table (T)),
- να βάψει την καρέκλα (Chair (Ch)),
- να ανοίξει την πόρτα (Door (D)).

Ο πράκτορας πρέπει:

- να επιλέγει κατάλληλα ποιον στόχο θα προσπαθήσει να επιτύχει κάθε φορά,
- να σχεδιάζει ένα πλάνο ενεργειών για τον επιλεγμένο στόχο,
- να λαμβάνει υπόψη του το κόστος των βημάτων και τη μεταφορά αντικειμένων.

Στην εργασία μελετάμε **δύο εκδοχές του περιβάλλοντος**:

- a) **Στατικό περιβάλλον**: Οι θέσεις όλων των αντικειμένων είναι σταθερές σε όλα τα επεισόδια.
- b) **Δυναμικό περιβάλλον**: Οι θέσεις των αντικειμένων T, Ch και D αλλάζουν τυχαία στην αρχή κάθε επεισοδίου (οι θέσεις των B, Cl, K, Cd παραμένουν σταθερές).

Και για τις δύο εκδοχές εκτελούμε πείραμα 100 επεισοδίων και συγκρίνουμε την αναμενόμενη χρησιμότητα, τον αριθμό βημάτων και το ποσοστό επιτυχίας.

2 Μοντελοποίηση Περιβάλλοντος

2.1 Πλέγμα και αρχική θέση πράκτορα

Το περιβάλλον είναι ένα ορθογώνιο πλέγμα:

$$5 \times 5$$

Ο πράκτορας ξεκινά πάντα από την αρχική θέση:

$$pos(1, 1)$$

και γνωρίζει ρητά το μέγεθος του πλέγματος μέσω της πεποίθησης:

$$grid_size(5, 5)$$

Επιπλέον, υπάρχει περιορισμός μεταφοράς αντικειμένων:

$$max_carry(3)$$

δηλαδή ο πράκτορας μπορεί να μεταφέρει ταυτόχρονα μέχρι τρία αντικείμενα.

2.2 Αντικείμενα, στατικές θέσεις και τοίχοι

Τα αντικείμενα που υπάρχουν στο περιβάλλον, μαζί με τις αρχικές τους θέσεις στην **στατική** εκδοχή, είναι:

Σύμβολο	Περιγραφή	Θέση (x,y)
B	Brush (Πινέλο)	(1,5)
Cl	Color (Χρώμα)	(5,5)
K	Key (Κλειδί)	(1,4)
Cd	Code (Κωδικός)	(3,5)
T	Table (Τραπέζι)	(5,1)
Ch	Chair (Καρέκλα)	(4,2)
D	Door (Πόρτα)	(3,1)

Πίνακας 1: Αντικείμενα περιβάλλοντος και οι αρχικές τους θέσεις (στατικό περιβάλλον).

Οπτική απεικόνιση του περιβάλλοντος:

5	B		Cd		Cl
4	K				
3					
2				Ch	
1			D		T
	1	2	3	4	5

Σχήμα 1: Agent Environment

Επιπλέον, το περιβάλλον περιλαμβάνει **τοιίχους** (walls), δηλαδή θέσεις στις οποίες ο πράκτορας δεν μπορεί να κινηθεί:

Τοίχος	Θέση (x,y)
wall(2,1)	(2,1)
wall(2,2)	(2,2)
wall(4,4)	(4,4)
wall(4,5)	(4,5)

Πίνακας 2: Θέσεις τοίχων στο πλέγμα.

Στο **στατικό περιβάλλον** οι θέσεις του Πίνακα 1 παραμένουν ίδιες για όλα τα επεισόδια.

Στο **δυναμικό περιβάλλον** οι θέσεις των T, Ch και D ενημερώνονται από το περιβάλλον στην αρχή κάθε επεισοδίου (μέσω percepts) και μπορούν να αλλάζουν, με την προϋπόθεση ότι:

- δεν συμπίπτουν μεταξύ τους,
- δεν τοποθετούνται πάνω σε τοίχους ή άλλα αντικείμενα,
- παραμένουν σταθερές κατά τη διάρκεια του επεισοδίου.

2.3 Ενέργειες πράκτορα (υλοποιημένες και αποκλίσεις)

Ο πράκτορας, σε επίπεδο προδιαγραφής, διαθέτει κινήσεις και ενέργειες χειρισμού αντικειμένων (grab/drop) καθώς και ενέργειες paint και open. Ωστόσο, κατά την υλοποίηση προέκυψαν ασυμβατότητες ανάμεσα στις εντολές που εκδίδονται από τον πράκτορα (ASL) και στις ενέργειες που αναγνωρίζει το Java περιβάλλον (executeAction).

ΣΗΜΕΙΩΣΗ:

Όλες οι ενέργειες είναι υλοποιημένες στο `main_agent.asl`, ωστόσο θα τις διαχωρίσουμε εδώ για να τις κάνουμε ξεκάθαρες στο report.

Υλοποιημένες ενέργειες στο Java περιβάλλον:

- `move(X,Y)`: κίνηση σε κελί με συντεταγμένες (1-based) X,Y ,
- `pick(O)`: παραλαβή αντικειμένου O από την τρέχουσα θέση,
- `paint(O)`: βάψιμο στόχου O (chair/table) υπό προϋποθέσεις,
- `open(door)`: άνοιγμα πόρτας υπό προϋποθέσεις,
- `noop`: καμία ενέργεια.

Αποκλίσεις/συγκρούσεις με τις εντολές ASL:

- Οι κινήσεις τύπου `move(up/down/left/right)` δεν αντιστοιχούν απευθείας σε ενέργεια του περιβάλλοντος, καθώς το περιβάλλον αναμένει `move(X,Y)`.
- Οι ενέργειες `grab(O)` και `drop(O)` δεν υποστηρίζονται στην τρέχουσα έκδοση του Java περιβάλλοντος (υπάρχει μόνο `pick(O)`).

Για τον λόγο αυτό, στην παρούσα έκδοση των πειραμάτων χρησιμοποιήθηκαν μόνο οι ενέργειες που εκτελούνται επιτυχώς από το περιβάλλον, ενώ οι υπόλοιπες προορίζονται για μελλοντική ολοκλήρωση.

2.4 Περιορισμοί και προϋποθέσεις εργασιών

Οι απαιτήσεις για κάθε εργασία/στόχο κωδικοποιούνται ρητά ως πεποιθήσεις:

```
//Task requirements
//To paint T or Ch, agent needs B(brush) and C1(color)
//To open D, agent needs K(key) and Cd(code)
needs_to_paint(T, [B,C1]).
needs_to_paint(Ch, [B,C1]).
needs_to_open(D, [K,Cd]).
```

Συνεπώς:

- Για να εκτελεστεί η `paint(T)` ή `paint(Ch)` ο πράκτορας πρέπει να κρατά τα αντικείμενα `B` (Brush) και `Cl` (Color).
- Για να εκτελεστεί η `open(D)` ο πράκτορας πρέπει να κρατά τα αντικείμενα `K` (Key) και `Cd` (Code).
- Ο πράκτορας μπορεί να μεταφέρει **μέχρι τρία** αντικείμενα ταυτόχρονα, όπως δίνεται από το `max_carry(3)`.

Οι ανταμοιβές (rewards) και οι ποινές (penalties) υλοποιούνται στην πλευρά του Java περιβάλλοντος, το οποίο ενημερώνει τη συνολική χρησιμότητα κάθε επεισοδίου.

3 Σχεδιασμός Πράκτορα BDI

3.1 Beliefs (Πεποιθήσεις)

Στον Jason, οι βασικές πεποιθήσεις του πράκτορα (αρχικές γνώσεις για τον κόσμο) είναι:

```
//Beliefs of agent

//Basic world info
pos(1,1). // agent starting position
grid_size(5,5). // 5x5 grid
max_carry(3). // agent can carry up to 3 objects

//Objects inside the environment
object(T, table).
object(Ch, chair).
object(D, door).
object(Cl, color).
object(Cd, code).
object(B, brush).
object(K, key).

//Task requirements
//To paint T or Ch, agent needs B(brush) and Cl(color)
//To open D, agent needs K(key) and Cd(code)
needs_to_paint(T, [B,Cl]).
needs_to_paint(Ch, [B,Cl]).
needs_to_open(D, [K,Cd]).

//Initial locations of movable objects
at(B, 1,5). // Brush
at(K, 1,4). // Key
at(Cd, 3,5). // Code
at(Cl, 5,5). // Color

//Initial locations of T, Ch, D (static version)
//Later, to make the system dynamic, these will come from percepts
```

```

at(Ch, 4,2). // Chair
at(D, 3,1). // Door
at(T, 5,1). // Table

//Walls location
wall(2,2).
wall(2,1).
wall(4,4).
wall(4,5).

//Rewards and Penalties are implemented in the java environment

```

Παράλληλα, ο πράκτορας διατηρεί δυναμικές πεποιθήσεις για τα αντικείμενα που ήδη κρατά και για τη συμβατότητα αντικειμένων:

```

//Utility predicates for the agent

//Compute how many objects the agent is currently carrying
carrying_count(N) :- .findall(O, have(O), L) & .length(L, N).

//Compatible
compatible(B).
compatible(Cl).
compatible(K).
compatible(Cd).
compatible(T).
compatible(Ch).
compatible(D).

//Incompatible if we carry something that is not declared compatible
incompatible(O) :- not compatible(O) & have(O).

```

3.2 Goals (Στόχοι)

Οι βασικοί στόχοι παραμένουν:

$$G = \{colored(T), colored(Ch), open(D)\}.$$

Στο επίπεδο BDI αυτοί χειρίζονται ως επιτεύξιμοι στόχοι. Ενδεικτικά, μπορούμε να ορίσουμε έναν αρχικό στόχο *start* που ζητά την επίτευξη και των τριών:

```

!mission

+!mission <- !achieve_colored(T);
               !achieve_colored(Ch);
               !achieve_open(D).

```

Στην πράξη, ο πράκτορας μπορεί να επιλέγει δυναμικά τη σειρά με βάση την αναμενόμενη χρησιμότητα κάθε στόχου.

3.3 Επιλογή στόχου βάσει Αναμενόμενης Χρησιμότητας

Για κάθε μη επιτευχθέντα στόχο $g \in G$ μπορούμε θεωρητικά να ορίσουμε την αναμενόμενη χρησιμότητα ως:

$$EU(g) = R(g) - Cost(Plan(g)),$$

όπου $R(g)$ είναι η θετική ανταμοιβή του στόχου (όπως υλοποιείται στο Java περιβάλλον) και $Cost(Plan(g))$ μια εκτίμηση του κόστους του πλάνου (αριθμός βημάτων $\times$ κόστος ανά βήμα).

Στην παρούσα υλοποίηση, ο πράκτορας **δεν** υπολογίζει ρητά την $EU(g)$ με αριθμητικό τύπο, αλλά ακολουθεί μια σταθερή σειρά στόχων στην εντολή `mission` ($T \rightarrow Ch \rightarrow D$). Η ανάλυση με αναμενόμενη χρησιμότητα χρησιμοποιείται κυρίως περιγραφικά, για να δείξουμε πώς θα μπορούσε να επεκταθεί το σύστημα σε μελλοντική δουλειά, όπου ο πράκτορας θα επέλεγε δυναμικά τον στόχο με την υψηλότερη $EU(g)$.

Η εκτίμηση του κόστους μπορεί να βασιστεί:

- στις αποστάσεις στο πλέγμα (μετρική Manhattan),
- στο αν ο πράκτορας ήδη κρατά τα απαιτούμενα αντικείμενα από τις σχέσεις `needs_to_paint / needs_to_open`,
- στον αριθμό αντικειμένων που μεταφέρει (`carrying_count/1`).

3.4 Σχεδιασμός Πλάνων Ενεργειών

Ο πράκτορας σχεδιάζει πλάνα της μορφής:

συλλογή απαραίτητων αντικειμένων $\rightarrow$ μετάβαση στον στόχο $\rightarrow$ εκτέλεση ενέργειας.

Συλλογή αντικειμένων

Η συλλογή πολλών αντικειμένων υλοποιείται με τη χρήση λίστας:

```
//Actions supported by the current Java environment
do(move(X,Y)). // move to absolute coordinates (1-based)
do(pick(0)). // pick up object 0 at current cell
do(paint(0)). // paint chair/table if conditions hold
do(open(door)). // open door if key+code and adjacency hold
do(noop). // no operation
```

Για παράδειγμα, ένα ενδεικτικό πλάνο για το `achieve_colored(T)` μπορεί να είναι:

```
// If Obj is already colored, do nothing
+!achieve_colored(Obj) : colored(Obj) <- true.

// If Obj is not colored yet, paint it
```



```

+!achieve_colored(Obj) : not colored(Obj) <- !paint(Obj).

// Goal: paint(Obj)
+!paint(Obj)
  : needs_to_paint(Obj, ReqList) // check required items (brush, color)
<- !collect_all(ReqList); // collect required items
  !go_to_obj(Obj); // move to the object's location
  do(paint(Obj)); // perform painting action (Java env)
  +colored(Obj). // update belief: Obj is now colored

```

Πλοήγηση και σχεδιασμός διαδρομής

Η πλοήγηση χρησιμοποιεί έναν στόχο `go_to(X,Y)` ο οποίος ζητά από το περιβάλλον να σχεδιάσει μία διαδρομή (π.χ. με A^*) και στη συνέχεια την ακολουθεί βήμα-βήμα:

```

//Navigation goals for the agent

//Optional high-level move goals (shortcut wrappers)
+!move_up <- do(move(up)).
+!move_down <- do(move(down)).
+!move_left <- do(move(left)).
+!move_right <- do(move(right)).

//Goal: go_to(X,Y)
//Use path planning ( $A^*$ ) from the current position to (X,Y)

//Already at target position -> nothing to do
+!go_to(X,Y) : pos(X,Y) <- true.

//Not there yet -> ask environment to plan a path, then follow that path step
  by step
+!go_to(X,Y) : pos(CX,CY) <- !plan_path(CX,CY,X,Y,Path);
               !follow_path(Path).

// Follow a path represented as a list of steps

+!follow_path([]) <- true.

+!follow_path([Step | Rest]) <- do_step(Step);
                               !follow_path(Rest).

```

Σημείωση υλοποίησης (τρέχουσα κατάσταση): Κατά την περίοδο συγγραφής της αναφοράς, η πειραματική εκτέλεση (στατικό και δυναμικό σενάριο) δεν κατέστη εφικτή λόγω τεχνικών σφαλμάτων ολοκλήρωσης ανάμεσα στον πράκτορα (ASL) και το περιβάλλον (Java). Συγκεκριμένα, εντοπίστηκαν conflicts σε επίπεδο αντιστοίχισης ενεργειών (actions) και φόρτωσης/εκτέλεσης του περιβάλλοντος, με αποτέλεσμα ο πράκτορας να μην μπορεί να ολοκληρώσει επεισόδια με αξιόπιστο τρόπο. Ως εκ τούτου, η ενότητα αυτή περιγράφει τον προτεινόμενο πει-

ραματικό σχεδιασμό, ενώ τα αριθμητικά αποτελέσματα και οι τελικές συγκρίσεις μεταφέρονται σε μελλοντική εργασία μετά την αποκατάσταση των σφαλμάτων.

4 Αποτελέσματα και Συζήτηση

4.1 Κατάσταση εκτέλεσης και διαθέσιμα ευρήματα

Η ποσοτική αξιολόγηση (100 επεισόδια για στατικό και 100 για δυναμικό περιβάλλον) δεν πραγματοποιήθηκε στην παρούσα έκδοση, καθώς η εκτέλεση του συστήματος δεν ήταν σταθερή λόγω σφαλμάτων ολοκλήρωσης agent–environment. Τα προβλήματα εντοπίζονται κυρίως:

- στην αντιστοίχιση των ενεργειών που εκδίδει ο πράκτορας (ASL) με τις ενέργειες που αναγνωρίζει το Java περιβάλλον,
- στη συνέπεια percepts/πεποιθήσεων μετά από αποτυχημένες ή μη αναγνωρισμένες ενέργειες,
- σε ζητήματα φόρτωσης/εντοπισμού της κλάσης περιβάλλοντος (classpath/package configuration) σε ορισμένες ρυθμίσεις.

Συνεπώς, στην παρούσα φάση η αξιολόγηση παραμένει **ποιοτική**: επιβεβαιώθηκε η ορθότητα του σχεδιασμού των στόχων, των πεποιθήσεων και της δομής των πλάνων, ενώ η πλήρης πειραματική σύγκριση θα παρουσιαστεί μετά την επίλυση των τεχνικών θεμάτων. **Περιορισμός:** Η παρούσα αναφορά εστιάζει στον σχεδιασμό και την υλοποίηση της αρχιτεκτονικής BDI και του περιβάλλοντος, όμως η τελική πειραματική επικύρωση (στατικό/δυναμικό σενάριο) δεν ολοκληρώθηκε λόγω τεχνικών bugs που απέτρεψαν σταθερή εκτέλεση επεισοδίων.

- **Διόρθωση bugs περιβάλλοντος και συνέπειας percepts:** Έλεγχος και διόρθωση σφαλμάτων που οδηγούν σε ασυμφωνία μεταξύ πραγματικής κατάστασης περιβάλλοντος και πεποιθήσεων πράκτορα, ειδικά μετά από αποτυχημένες ή μη εκτελεσμένες ενέργειες.
- **Σταθεροποίηση μηχανισμού πλοήγησης (A*):** Αν και ο αλγόριθμος εύρεσης διαδρομής (A*) λειτουργεί σωστά και επιστρέφει έγκυρη λίστα κινήσεων, δεν κατέστη δυνατό να εντοπιστεί εγκαίρως το πρόβλημα στην εκτέλεση των βημάτων της διαδρομής από τον πράκτορα. Ως αποτέλεσμα, ο πράκτορας παρέμενε στάσιμος, παρότι είχε υπολογιστεί επιτυχώς μονοπάτι προς τον στόχο. Η πλήρης διάγνωση και αποκατάσταση του συγκεκριμένου σφάλματος αποτελεί αντικείμενο μελλοντικής εργασίας.

4.2 Ζητήματα υλοποίησης που επηρέασαν την εκτέλεση

Κατά την ολοκλήρωση του συστήματος εντοπίστηκαν προβλήματα που δεν επέτρεψαν την ομαλή εκτέλεση επεισοδίων:

- **Διόρθωση bugs περιβάλλοντος και συνέπειας percepts:** Οι βασικές ενέργειες του πράκτορα (π.χ. κίνηση, συλλογή αντικειμένων, εκτέλεση στόχων) λειτουργούν σε λογικό επίπεδο, ωστόσο εντοπίστηκαν προβλήματα στην επικοινωνία και στον συγχρονισμό με το γραφικό περιβάλλον. Τα προβλήματα αυτά οδηγούν σε ασυμφωνία μεταξύ της πραγματικής κατάστασης του περιβάλλοντος, της οπτικής απεικόνισης και των πεποιθήσεων του πράκτορα, ιδιαίτερα μετά από ακολουθίες ενεργειών.
- **Σταθεροποίηση μηχανισμού πλοήγησης (A*):** Ο αλγόριθμος εύρεσης διαδρομής (A*) λειτουργεί σωστά και επιστρέφει έγκυρη λίστα κινήσεων προς τον στόχο. Παρόλα αυτά, δεν κατέστη δυνατό να εντοπιστεί εγκαίρως το πρόβλημα που προέκυπτε κατά την εκτέλεση των βημάτων της διαδρομής σε συνδυασμό με το γραφικό περιβάλλον. Ως αποτέλεσμα, ο πράκτορας παρέμενε στάσιμος στην οπτική αναπαράσταση, παρότι είχε υπολογιστεί επιτυχώς μονοπάτι. Η πλήρης διάγνωση και αποκατάσταση του συγκεκριμένου ζητήματος αποτελεί αντικείμενο μελλοντικής εργασίας.